

Tenet: Agent Memory as a Self-Consistent Belief State

Bi-temporal supersession, belief-evidence consistency, and LLM-free reads
for long-horizon agents over changing knowledge

Anas

<https://github.com/Nas01010101/tenet>

July 2026

Abstract

Long-term memory for LLM agents is almost universally implemented as *retrieval over a growing log of past turns*. We argue this is the wrong abstraction for an agent that must model a *changing* world, and we characterize the failure as **knowledge churn**: as a fact is updated N times over a long interaction, stale versions crowd the retrieval budget k , and once $N > k$ the current value is no longer reliably retrieved. On a controlled benchmark a strong RAG memory falls from 100% to 50% current-value accuracy as a fact is updated $2 \rightarrow 12$ times—under both a **gpt-4o-mini** and a **gpt-4o** reader, so the collapse is structural, not a reader artifact. We propose TENET, which reframes memory as a **self-consistent belief state**: (i) turns are distilled into atomic facts with stable semantic keys; (ii) a **bi-temporal** record lets a changed fact *supersede* its predecessor at ingestion (history kept, time-travel queryable); (iii) a **belief-evidence consistency rule** retires raw evidence that echoes a superseded belief; (iv) a **surprise-gated write policy** stores only observations the current state does not already predict; and (v) **belief-anchored evidence expansion** spends spare context on raw turns from the sessions the belief state already surfaced. The read path never calls an LLM. TENET holds 100% on this templated single-attribute churn primitive at every update level (§6.1); on the harsher *paraphrased* multi-attribute ChurnBench (§6.8) it was initially *falsified* (worst arm); a read-time consistency rule plus write-side embedding key-resolution—both defaulted on—lift its churn half-life from <2 to 32 ($U=32 \approx 82\text{--}100\%$ across runs, reader non-determinism): at best this *ties* delete-outright Mem0-style (flat 100) without beating it on raw accuracy, while keeping LLM-free reads. On MemoryAgentBench FACTCONSOLIDATION (ICLR 2026) it scores 86.5% single-hop [82.8, 89.5]—above the published **gpt-4o-mini**-tier state of the art (78.0) despite using a local 7B backbone—and ties the multi-hop state of the art (30.2); on MemoryAgentBench ACCURATE RETRIEVAL it averages 59.3, second only to HippoRAG-v2 (65.1) among published systems while being the only one whose ingestion never calls an LLM, and beats the field on EventQA (70.7 vs 67.6; Wilson CI excludes). All numbers are reproducible from released code with official metrics and prompts verbatim.

1 Introduction

An agent that talks to a user for months does not need a transcript; it needs a *model of the user* that stays true as the user changes. Yet the dominant memory design—retrieval-augmented generation over stored turns [1, 2]—treats memory as a document index: embed every turn, retrieve the top- k most similar at query time, and let the reader sort it out. This works for one-shot recall of a *static* fact. It fails, quietly, when facts *change*.

We formalize this failure as **knowledge churn**. A user who moves cities several times generates many “I moved to X ” turns; all are similar to the query “where do I live?”, so the top- k fills

with *stale* versions. Once the number of updates N exceeds the budget k , the latest value may not be retrieved at all; even when it is, the reader must infer recency from a pile of contradictory statements. Accuracy collapses (§6.1), and—critically—a stronger reader cannot rescue it, because the failure occurs before the reader sees anything.

The root problem is the abstraction. A document store has no notion that “I live in Boston” and “I live in Seattle” are the *same fact* with a *changed value*; both are just passages. We argue memory should instead be a **belief state**: a compact set of *current* beliefs about the world, each with a temporal extent, updated by observation, kept internally consistent, and queryable across time. This is the stance predictive-coding accounts take toward perception [22]; we bring it to agent memory and pay particular attention to *where* consistency is enforced: at ingestion, so the store never contains a stale current value, rather than at assembly time, where every read must re-derive it.

Contributions.

1. We identify and name **knowledge churn**, a structural failure mode of retrieval-augmented memory, give a controlled benchmark exhibiting it (RAG: 100% $\rightarrow$ 50% as a fact is updated $2 \rightarrow 12\times$; reader-invariant), and show a belief-state memory is immune on this templated primitive (§6.1)—then stress-test the claim on a harder paraphrased, multi-attribute variant (ChurnBench, §6.8) that falsifies it and motivates the read-time consistency fix (§6.9).
2. We present TENET: bi-temporal keyed supersession at ingestion, a **belief–evidence consistency rule** that retires stale raw evidence, a **surprise-gated write policy**, salience-decay forgetting, and budget-bounded **belief-anchored evidence expansion**—in a light, graph-free store whose read path never calls an LLM (§4).
3. On the standardized **MemoryAgentBench** [7] we evaluate both core competencies with official metrics and prompts verbatim, Wilson CIs, and matched-reader controls. FACTCONSOLIDATION: single-hop 86.5 [82.8, 89.5], above the published mini-tier SOTA (78.0 [9], CI excludes) on a *weaker* (local 7B) backbone; multi-hop 30.2, tying the same SOTA, where every system in the original benchmark table scores ≤ 7 . ACCURATE RETRIEVAL: average 59.3, second among published systems to HippoRAG-v2’s 65.1 [8]—which runs LLM OpenIE over every context token, where TENET ingests with embeddings only—and ahead of the field on EventQA (70.7 vs 67.6, CI excludes) (§6.5).
4. We reimplement four published memory methods (Mem0-style consolidation, CAR aggregation [9], a HippoRAG-v2-style OpenIE+PPR graph, and MemAgent-style [16] overwrite memory) as arms in the *same* harness with the *same* backbone, isolating the memory mechanism as the only variable (§6.6).

2 Related work

Retrieval memory. Mem0 [1] distills salient facts at write time over a vector store with entity links; it attaches only a *creation* timestamp and notably removed its graph variant after finding it $3\times$ slower for a thin gain—evidence we take seriously in choosing a light vector substrate. LongMemEval [2] is the standard long-horizon benchmark; its V2 [3] adds a latency-aware metric, signalling a field shift toward accuracy *per cost*, which our per-token results target.

Temporal knowledge graphs. Zep/Graphiti [4] maintains a *bi-temporal* knowledge graph (valid + transaction time) with edge invalidation—the closest prior system in temporal semantics. It

differs in substrate and mechanism: invalidation is LLM-mediated at graph-construction time over (subject, predicate, object) edges, reads traverse a graph, and neither a belief–evidence consistency rule over raw text nor surprise gating is present. TENET keeps the bi-temporal semantics without the graph, and its supersession is deterministic given the distilled key.

The 2026 bi-temporal convergence. Concurrently with this work, several systems adopted bi-temporal supersession: MemStrata [10] applies a deterministic (subject, relation, object) supersession rule over a bi-temporal ledger with no LLM in the read path—and shows *similarity-threshold* supersession leaks stale values where deterministic keying does not, independently corroborating our keyed design; Engram [11] pairs a bi-temporal knowledge graph with a hybrid facts-plus-raw-chunks read path (converging on our dual-pool finding); TOKI [12] gives contradiction resolution a formal bitemporal operator algebra. On conflict-resolution benchmarks, [9] shows the *assembly* step dominates: deterministic max(serial) aggregation after retrieval sets the published SOTA on MemoryAgentBench FactConsolidation. TENET differs from all of these in *where* consistency is enforced—at ingestion (the store never contains a stale current value) *and* at recall (belief–evidence consistency retires stale raw evidence)—and in coupling the belief state to a write policy (surprise gating) and a budget-bounded evidence expansion, evaluated on the knowledge-churn axis none of them report.

Memory in the Qwen ecosystem. Alibaba’s 2026 memory line takes the opposite design point: QwenLong-L1.5 [13] reaches 76.4 on LongMemEval by RL-training a 30B reasoner that reads up to 128K tokens per query; AgeMem [14] RL-trains the memory policy itself; ActMem [15] (Alibaba-co-authored) reaches 75.6 with LLM-built causal graphs at ingestion. All three spend heavily—at training time, ingestion time, or read time. TENET reaches 60.0 on the same benchmark at the same reader tier with zero-LLM ingestion and $\sim 2\text{K}$ read tokens per query ($\approx 79\%$ of QwenLong-L1.5’s score at under 2% of its read budget), and its structural wins (churn, conflict resolution) survive on a 7B backbone.

Active memory navigation. A concurrent 2026 line treats memory access itself as an agentic, learned process. NapMem [17] exposes a four-level memory pyramid (raw conversation / typed records / topic tracks / user profile) as a structured action space and trains a GRPO policy that descends the pyramid and stops once evidence is sufficient—a trained 9B policy beats an untrained 397B model on memory benchmarks. QwenLong-L1.5 [13] additionally plans a navigational path over memory chunks, lifting LongMemEval by +15.6 points. MemFlow [18] is training-free: an LLM router selects a memory tier per query, and an LLM Validator retries with a heavier tier on a low-confidence read. MemReader [19] moves the same active-navigation idea to the write side, extracting memory rather than passively logging it. AgeMem [14] and Mem- α [20] RL-train the read/construction policy itself; MemCog [21] reports a similar cognitive-navigation frame (affiliation unverified). All of these make the *read* path agentic: an LLM or an RL policy decides how deep to go and when to stop. TENET takes the opposite bet on the read side—it keeps recall LLM-free and gets adaptive depth from an embedding saturation gate (§4.8) instead of a learned or LLM-mediated stop.

OS-style and agentic memory. MemGPT/Letta [5] pages memory between a context “RAM” and archival “disk”; A-Mem [6] lets the agent link and evolve notes. Both are largely append-oriented: fact supersession, consistency between distilled and raw layers, and forgetting are not first-class operations.

What is missing. No prior system combines (a) ingestion-time bi-temporal keyed supersession, (b) an explicit belief–evidence consistency rule spanning the distilled and raw layers, (c) surprise-gated writes, (d) principled forgetting, and (e) belief-anchored budget-bounded expansion in a

graph-free store with an LLM-free read path—nor does any report the knowledge-churn regime.

3 Knowledge churn

Let a fact with key κ take values $v_1, \dots, v_N$ over an interaction, each update producing at least one stored turn. A flat retriever with budget k ranks turns by query similarity; all N versions are near-equally similar to a query about κ (paraphrases of the same statement), so the expected number of *stale* passages in the top- k grows with N , and for $N > k$ the *current* value v_N competes with $N - 1$ distractors for k slots with no ranking signal that prefers it. Two consequences: (1) retrieval of v_N becomes unreliable; (2) even when retrieved, the reader receives contradictory values and must infer recency from text alone. Both are properties of the *store*, not the reader—which predicts the reader-invariance we observe in §6.1. A belief state removes the problem at the source: the store contains exactly one current value per key, so retrieval of the current value is $O(1)$ in N .

4 Method

TENET stores two layers over one bi-temporal table: a **belief layer** $\mathcal{B}$ of distilled, keyed facts and an **evidence layer** $\mathcal{E}$ of raw turns. Writes maintain consistency; reads are deterministic lookups plus embedding search—no LLM call.

4.1 Distillation into keyed beliefs

Each turn is distilled by a small LLM into atomic facts, each with a stable semantic key $\kappa = \text{subject}::\text{attribute}$ (e.g. `user::residence`), a salience $s \in [0, 1]$, and an event time. The key is what makes supersession reliable: embedding similarity cannot separate a *restated* fact from a *value-changed* one (we measure a value change at cosine 0.99—*higher* than a plain rephrasing of the same fact at 0.79, so no similarity threshold separates a changed value from a restatement), but a shared key can. Where the source text is already structured (as in benchmark fact streams), keys are computed deterministically from the text with no LLM at all (§6.4).

4.2 Bi-temporal supersession

Every memory carries event time (`valid_at`, `invalid_at`) and transaction time (`created_at`, `expired_at`). Storing a fact with key κ whose value differs from the current fact at κ *supersedes* it: the old fact’s `invalid_at`/`expired_at` are set; it leaves the current set but remains in history. Current recall filters `expired_at` IS NULL; *recall(as_of = t)* reconstructs the belief set as of any past t (time-travel). Supersession is thus enforced *at ingestion*: the current store never contains a stale value, and no per-read aggregation is needed to re-derive freshness.

4.3 Belief–evidence consistency

The evidence layer lets the reader answer detail questions, but it is also where stale values hide: a raw turn “I moved to Boston” survives even after the belief `user::residence` moves on. We therefore retire from current recall any raw slice e whose embedding is close to a *superseded* belief:

$$\text{exclude } e \Leftrightarrow \max_{f \in \mathcal{B}_{\text{expired}}} \cos(\phi(e), \phi(f)) \geq \tau_{\text{stale}}, \quad \tau_{\text{stale}} = 0.80. \quad (1)$$

This single rule turns supersession from a fact-layer nicety into end-to-end correctness: it takes current-value accuracy from 55% to 100% (§6.3).

4.4 Surprise-gated writes (predictive coding)

Predictive coding stores *prediction error*, not everything. On write, a raw observation e is discarded if the store already predicts it, i.e. it is near-identical to an existing slice:

$$\text{store } e \Leftrightarrow \max_{e' \in \mathcal{E}} \cos(\phi(e), \phi(e')) < g_{\text{surprise}}, \quad g_{\text{surprise}} = 0.97. \quad (2)$$

This bounds the store and drops redundant repetition with no accuracy loss (§6.3).

4.5 Forgetting and dual-pool recall

Each memory’s rank is relevance times a decay factor $d = 2^{-\Delta t/h} \cdot (1 + \beta \log(1 + \text{uses})) \cdot (0.6 + 0.8s)$ with half-life $h = 14\text{d}$; a sweep archives current, unpinned memories with d below a threshold; pinned identity facts never decay. Retrieval is a **dual pool**—beliefs for consistency, evidence for verbatim detail—guaranteeing each a share of the budget.

4.6 Fact dynamics: learning how facts change

A belief state gains a drift model when it can predict its own staleness. The bi-temporal ledger is a training set for exactly this: every superseded fact is an observed lifetime (`valid_at` $\rightarrow$ `invalid_at`) and every current fact a right-censored one. Per key *class* c (the attribute part of κ), we place a conjugate Gamma prior on an exponential change rate; with n_c observed supersessions and total exposure T_c , the posterior-predictive survival is Lomax:

$$P(\text{still valid after } \Delta t \mid c) = \left(\frac{\beta_0 + T_c}{\beta_0 + T_c + \Delta t} \right)^{\alpha_0 + n_c}, \quad (3)$$

so “residence” learns a slow hazard and “mood” a fast one, per user, in closed form, with no LLM. A second ledger statistic captures *ripple*: the smoothed co-supersession probability $P(c' \text{ changes} \mid c \text{ changed})$; when a correlated key changes, the survival of its neighbors is discounted ($S \rightarrow S^{1+\gamma \text{ bump}}$), so one observation updates beliefs about unobserved attributes—one observation updating beliefs about many keys. Both surface as a *confidence* on every recalled fact and an *uncertain_facts()* list the agent can use to re-verify with the user. Deliberately, confidence does **not** demote a fact’s rank: a doubted fact is still the best known answer, and we measured rank-demotion re-creating the churn failure of §6.1 (current value pushed behind distractors) before reverting to annotation-only. A pre-registered follow-up asked whether the same confidence could route *reader compute* (extractive / cheap / full tiers): it cannot—on 120 questions, no threshold configuration in an 84-point sweep saved reader tokens within 2pp of the 91.7% all-full-reader baseline, because confidence is a *currency* signal orthogonal to the relevance errors that dominate extractive mistakes (rank-1 belief correct on currency but wrong on attribute in a third of routed cases). Calibration is sufficient for caveats and doubt surfacing; not for compute routing. We report both nulls rather than shipping either.

A trained neural dynamics module. The closed-form model is memoryless by family; we also train a compact (276k-param, 1-layer GRU) *neural temporal point process* over fact-update events, whose Weibull hazard head admits non-constant per-key hazards and whose auxiliary heads predict the next key to change (learned ripple) and the next value (contrastively, over embeddings). On a controlled synthetic corpus with planted periodic, bursty, and cascading structure—provably outside the closed-form family—the neural model reduces held-out NLL from 2.76 to 1.99 nats/event (paired-bootstrap gain 0.81, 95% CI [0.70, 0.93]), halves survival mis-calibration (KS 0.56 $\rightarrow$ 0.26), and predicts the next value at 0.997 recall@5 vs 0.05 chance, stable over five seeds. On real

counterfactual-update chains the result is deliberately mixed: better calibration and observed-lifetime likelihood on FactConsolidation, but the censoring-dominated aggregate NLL loses to a marginal-rate baseline, and LongMemEval yields too few clean chains ($n=21$) to conclude. The exported model is a 1 MB numpy artifact evaluated in 106 μ s, opt-in behind an env flag with default behavior unchanged, and passes the churn-integrity guard.

4.7 Belief-anchored evidence expansion

Compressing a session into a few keyed beliefs wins churn and efficiency but can drop the fine detail a multi-hop question needs, even when the right session *is* retrieved. We recover it without reverting to flat retrieval: the top- k dual-pool result names both the belief state and the sessions it came from. Given spare context budget B , TENET fills it with additional query-relevant raw turns drawn *only from those already-surfaced sessions*, subject to the same consistency filter (§4.3) so no stale value re-enters. B is set to the baseline RAG budget, so expansion never spends more context than flat retrieval. The knob traces an accuracy–efficiency frontier: $m = 0$ is the efficiency point; m large under budget B is the parity point.

4.8 Saturation-gated navigation

Belief-anchored expansion (§4.7) draws raw evidence from a session boundary; associative multi-hop recall goes further by re-conditioning the cue on retrieved evidence and re-scoring the store, but at a caller-fixed depth $\text{hops} = N$. A fixed N over-fetches simple queries (the extra hops add distractors that hurt the reader) and under-fetches genuinely multi-hop ones. `navigate()` replaces the fixed depth with an iterative, budget-bounded hop-deepening loop that reuses belief-anchored expansion at every hop and adopts hop $d+1$ only while its newly-reached evidence is still informative:

$$\text{adopt hop } d+1 \iff \max_{e \in \text{new}_{d+1}} \text{score}(e) \geq \tau_{\text{gain}}, \quad \tau_{\text{gain}} = 0.15, \quad (4)$$

where new_{d+1} is the evidence reached at depth $d+1$ but not at depth d . When no hop clears the gate, or the hop budget (≤ 4 , mirroring NapMem’s tool-call budget) is exhausted, navigation stops and returns the last adopted pool, dropping the marginal hop’s distractors rather than keeping them. The gate is an embedding-only marginal-relevance test over scores TENET already computes for recall, so it adds no LLM call and requires no training: it is the training-free counterpart to NapMem’s learned stop and to MemFlow’s LLM-Validator retry, running at embedding-search latency ($\sim$ ms) instead of an LLM round trip.

We validate the mechanism, not a benchmark number. On a deterministic, torch-free synthetic store (hashed bag-of-words embeddings, no network call), `navigate()` (a) reaches a bridge fact invisible to broad top- k retrieval via an associative hop, and (b) stops early (hop 2) on a saturated single-hop query while continuing to hop 3 on a genuinely multi-hop one, confirming the gate behaves as intended (multi-hop reach, single-hop early-stop). A subsequent seeded A/B on FACTCONSOLIDATION 6k cells at the `qwen3.7-plus` tier ($n=20$ per cell, same reader, prompts, and seed; retrieval-pool construction the only variable) measured *no benefit*: accuracy identical to default recall on both cells, and -5 to -10 pp against a fixed-4-hop arm, all within Wilson CIs. Dumped trajectories confirm the mechanism executed as designed (pools differed from baseline in 21/21 rows, adaptive depth 2–3): a sufficiently strong reader is robust to the larger fixed pool, so adaptive early-stopping buys no answer precision at this tier. We therefore report navigation as a mechanism contribution only, with a measured null at the tested tier.

5 Experimental setup

Benchmarks. (1) A controlled **knowledge-churn** benchmark (one fact updated N times amid distractors); (2) a controlled **knowledge-update** set for ablations; (3) **LongMemEval_S** [2] ($\sim 115\text{K}$ -token histories); (4–5) both core competencies of **MemoryAgentBench** [7] (ICLR 2026): FACTCONSOLIDATION (conflict resolution; all 800 questions) and ACCURATE RETRIEVAL ($\sim 2,000$ questions over 22 contexts of 197K–534K tokens).

Protocol fidelity. For MemoryAgentBench we use the official metrics and prompts *verbatim* from the benchmark code: SubEM for FactConsolidation and RULER-QA; multiple-choice accuracy for EventQA; the official LLM-judge templates for LongMemEval(S*). Readers are matched to the published tier being compared (**gpt-4o-mini** for AR; a deliberately *weaker* local **qwen2.5:7b** for FC). Every TENET number is paired with a naive-RAG control sharing the reader, chunking, and embedder, so the memory mechanism is the only variable.

Integrity rules. API failures are excluded, never scored as wrong (0 exclusions in FC; exclusions reported where they occur); abstentions are scored wrong; ingestion caches are keyed by the hash of the exact chunk list; we report Wilson 95% CIs and dump every miss for error analysis. All numbers reproduce from single documented commands.

6 Results

6.1 Knowledge churn

One fact updated N times amid distractors, $k = 6, 12$ principals per point:

updates N	2	4	6	8	10	12
RAG	100	100	100	67	58	50
TENET	100	100	100	100	100	100

RAG degrades monotonically once $N > k$ (-50 pp); TENET is flat at 100%. The curves are *identical* under a **gpt-4o-mini** and a **gpt-4o** reader: once $N > k$ the latest value is not reliably retrieved, so a stronger reader cannot rescue RAG—the failure is structural, exactly as §3 predicts.

6.2 LongMemEval_S: the accuracy–efficiency frontier

$n=40$, $k=10$, **gpt-4o** reader (the reader Mem0 and Zep report against), local embedder; baselines run under identical settings:

System	mode	recall@10	QA acc	reader tok	acc/1k tok
full-context	—	—	$\sim 65\%^*$	$\sim 124,000$	0.5
RAG	top- k turns	95%	57.5%	2,101	27.4
TENET	efficiency ($m=0$)	97.5%	52.5%	1,067	49.2
TENET	parity (expansion)	97.5%	57.5%	2,083	27.6

At the efficiency point TENET answers at half the context for the best accuracy-per-token of any system ($1.6\times$ RAG, $\sim 100\times$ full-context) at recall parity. Belief-anchored expansion, capped at RAG’s own budget, brings one-shot accuracy to *parity* ($57.5 = 57.5$) at fewer tokens. On a **gpt-4o-mini** reader the parity point edges ahead (60.0 vs 55.0); the per-token dominance holds across the

`gpt-4o-mini` and `gpt-4o` readers we ran ($\approx 1.6\times$). Given reader stochasticity of $\approx \pm 5\text{--}7$ pp at $n=40$, we report one-shot accuracy as parity, not a win. The one category still behind RAG is multi-session synthesis (42.9 vs 57.1, up from 28.6; §8).

Reader-generality at the frontier tier. We subsequently ran the identical captured task set (fresh $n=40$ sample, seed 0, Qwen distiller) through three 2026 frontier readers from independent families under one cross-family judge (`qwen3.7-plus`). An initial pass batched 10 questions per CLI call; a methodology audit flagged this as contaminable, so we **re-ran two scriptable readers one call per item** (`gpt-5.5`, `gemini-3.5-flash`) and re-judged—these un-batched numbers are of record. Batching had inflated absolute accuracy by $\sim 2\text{--}17$ pp; the *directional* claims survive the clean measurement: $\text{TENET} \geq \text{RAG}$ at both operating points on both readers (77.5/77.5 vs 75.0 GPT-5.5; 75.0/72.5 vs 70.0 Gemini), the efficiency point delivers $\approx 1.9\times$ accuracy-per-token at half the context, and the multi-session weakness still *inverts* un-batched (RAG 50.0 vs TENET 62.5 on both). Per-cell Wilson CIs overlap at $n=40$; the evidence is the directional replication across two independently-isolated reader families. Artifacts: `docs/lme_unbatched*.jsonl`.

6.3 Ablations

On the controlled knowledge-update set, removing the belief–evidence consistency rule (§4.3) drops current-value accuracy from 100% to 55% with a 45% stale-leak rate; restoring it recovers 100%/0%. It is the single most important mechanism. Surprise gating discards 15% of observations as redundant with no accuracy change, bounding the store where RAG grows without bound.

6.4 MemoryAgentBench FactConsolidation

Serial-numbered facts with counterfactual updates; questions require the *current* value. SubEM and the official reader prompt verbatim; all 800 questions; Wilson 95% CIs; 0 API-error exclusions. TENET’s ingestion here is **zero-LLM**: supersession keys are computed deterministically from the fact text, so ingestion costs embeddings only, and the reader is a local `qwen2.5:7b`—below the published mini tier.

cell	naive-RAG	TENET	SOTA mini ¹	SOTA gpt-4o ¹
SH 6K	36.0	89.0 [81.4, 93.7]	71	99
SH 32K	50.0	91.0 [83.8, 95.2]	78	92
SH 64K	52.0	85.0 [76.7, 90.7]	81	95
SH 262K	53.0	81.0 [72.2, 87.5]	82	93
SH pooled	47.8	86.5 [82.8, 89.5]	78.0	94.8
MH 6K	5.0	42.0 [32.8, 51.8]	34	57
MH 32K	3.0	30.0 [21.9, 39.6]	27	50
MH 64K	3.0	29.0 [21.0, 38.5]	33	58
MH 262K	7.0	20.0 [13.3, 28.9]	27	41
MH pooled	4.5	30.2 [26.0, 34.9]	30.2	51.5

¹[9]: candidate extraction + max(serial) aggregation.

Single-hop 86.5 exceeds the published mini-tier SOTA (78.0; our CI excludes it) on a weaker backbone; every system in the original MAB table scores ≤ 60 (Zep 7, Mem0 18, MemGPT 28). Multi-hop exactly ties the mini-tier SOTA, where the original table’s best is ≤ 7 . Single-hop barely degrades with haystack length ($89 \rightarrow 81$ from 6K $\rightarrow$ 262K) because the store is conflict-resolved at ingestion—the property assembly-time aggregation must re-derive at every read. Multi-hop still

degrades with length ($42 \rightarrow 20$); reported honestly. The identical reader over naive-RAG memory scores 47.8/4.5: the mechanism, not the reader, carries the result.

6.5 MemoryAgentBench Accurate Retrieval

22 contexts of 197K–534K tokens, four sub-benchmarks, official per-benchmark metrics, matched `gpt-4o-mini` reader. TENET ingestion is again zero-LLM (date-aware structured chunks + embeddings only):

sub-benchmark (official metric)	naive-RAG	TENET	HippoRAG-v2 ¹
RULER SH-QA (SubEM, $n=100$)	74.0	75.0	76
RULER MH-QA (SubEM, $n=100$)	40.0	45.0	66
LongMemEval(S*) (LLM-judge, $n=300$)	46.0	46.3	50.7
EventQA (choice acc, $n=1500$)	71.3	70.7 [68.3, 72.9]	67.6
AR average	57.8	59.3	65.1

¹Strongest AR system in the MAB paper. Other published frameworks: Mem0 32.6, Zep 37.5, MemGPT ≈ 39 .

TENET beats the published field on EventQA (70.7 vs 67.6; CI excludes), reaches parity on RULER SH-QA, and is second on AR average—behind only HippoRAG-v2, whose ingestion runs LLM OpenIE over every token of the 197K–534K contexts, where TENET never calls an LLM at ingestion. RULER MH-QA is the honest loss (45 vs 66): Personalized-PageRank graph traversal is genuinely stronger at multi-hop chaining over narrative text. LME(S*) is 4.4 points short after five documented retrieval iterations; the residual misses are answer-synthesis-bound (both arms identical), not retrieval-bound.

6.6 Same-harness reproductions of published methods

To remove any backbone confound, we reimplement four published memory mechanisms as arms of the *same* FactConsolidation harness with the *same* local-7B reader and embedder: CAR (max(serial) candidate aggregation [9]), Mem0-style LLM consolidation (batched ADD/UPDATE against nearest neighbors [1]), a HippoRAG-v2-style graph (OpenIE triples, synonym edges, Personalized-PageRank blended with dense scores [8]), and MemAgent-style overwrite memory (question-conditioned rolling rewrite [16]). These are mechanism reproductions at matched backbone, not the authors’ full systems. On the 6K+32K cells ($n=200$ pooled per axis; MemAgent subsampled $n=25$ at 6K only, as it reads the full haystack per question):

6K+32K pooled	TENET	CAR	Mem0-style	HippoRAG-style	MemAgent-style
SH	90.0 [85.1, 93.4]	87.5 [82.2, 91.4]	81.0	66.0	44.0
MH	36.0 [29.7, 42.9]	33.0 [26.9, 39.8]	12.0	9.0	16.0

TENET leads every reproduced arm on both axes. CAR—the published-SOTA recipe—is closest (CIs overlap): ingestion-time supersession matches or beats the best assembly-time aggregation while doing the consistency work once at write time instead of at every read. Mem0-style consolidation holds up single-hop (81.0) but collapses multi-hop (12.0); the graph arm is limited by 7B-quality OpenIE (66.0/9.0); overwrite memory loses information by construction (44.0/16.0).

6.7 Case study: web-agent trajectory memory (LongMemEval-V2)

LongMemEval-V2 [3] shifts the setting from chat to *web-agent trajectory* histories (up to 115M tokens; DOM states, actions, URLs) and scores a latency–accuracy frontier (LAFS) against a strong RAG reference (51.0% accuracy) using a mandated Qwen3.5-9B reader. We adapted TENET’s ingestion to this regime as a stress test of the retrieval substrate. A naive port (bge-small dense over trimmed DOM slices) retrieved the gold evidence for only $\sim 12\%$ of the 295 deterministic-metric questions. Three LLM-free changes rebuilt it: structure-aware chunking that keeps (action, URL, salient DOM) units together and dedups cross-state DOM; query cleaning; and hybrid retrieval—Qwen3-Embedding dense fused with BM25 by reciprocal-rank fusion. Gold-evidence recall rose to **59.7%** at a 48K reader budget (63.3% at 72K) against a measured corpus ceiling of $\sim 92\%$, with query latency ~ 0.25 s—the fast corner of the LAFS frontier. End-to-end accuracy proved *reader-gated*, not retrieval-gated—and the gate is the *reasoning mode*, not precision: a 4-bit local reader scores 45.1% [39.5, 50.8], an 8-bit one 40.3%, and the full-precision hosted reader *without* extended thinking 40.7% (it over-abstains), all pinned near the same $P(\text{correct} \mid \text{gold retrieved}) \approx 0.5$ – 0.6 extraction ceiling. The leaderboard’s default—extended thinking, ~ 8 K reasoning tokens per question—is what converts retrieval headroom to accuracy, and lies outside our compute budget. The case study supports the paper’s central claim in a second modality: the memory substrate—LLM-free ingestion and sub-second reads—transfers, and where the system falls short the binding constraint is measurable and external to the memory.

6.8 ChurnBench: a parametric stress test that falsifies our own churn claim

§6.1’s churn result uses one templated fact, updated up to 12 times amid distractors—a regime pre-registered to structurally favor supersession as soon as updates exceed k . ChurnBench generalizes it into a harness: a parametric generator sweeps updates-per-fact $U \in \{2, 4, 8, 16, 32\}$ over five keyed attributes (residence/job_title/car/phone/gym), each updated via *paraphrased*, first-person conversational statements (not templated serial-numbered lines) interleaved with distractor sessions; SubEM scoring; real `Tenet.ingest_session` (no hand-tuned keys). Four arms at matched backbone (qwen3.7-plus reader, bge-small embedder): TENET, RAG, and Mem0-/HippoRAG-v2-style reused verbatim from the §6.6 reproduction harness. Pre-registered gate: TENET churn half-life (largest swept U with accuracy $\geq 90\%$) $\geq 2\times$ the best baseline, CI-separated at $U=8$; $n=50$ questions/point, Wilson 95% CIs.

U	2	4	8	16	32
RAG	96.0	100.0	100.0	78.0	30.0
HippoRAG-v2-style	100.0	100.0	100.0	78.0	30.0
Mem0-style	90.0	98.0	100.0	100.0	100.0
TENET	62.0	44.0	44.0	44.0	46.0

Result: falsified, not a partial miss. Churn half-life: $\text{TENET} < 2$, RAG 8, HippoRAG-v2-style 8, Mem0-style 32. TENET is the *worst* of four arms at every tested U , including $U=2$ (its 95% CI excludes every baseline at $U \in \{2, 4, 8\}$)—the opposite of §6.1’s result and of the pre-registered hypothesis. Diagnosing a failing case by inspecting `recall()` directly shows the correct current fact *is* ranked first, but two or three raw turns for already-superseded values are still in the $k=10$ pool alongside it: the write-time stale-echo filter (§4.3, tuned for near-verbatim echoes) does not reliably retire a raw turn phrased differently from its distilled paraphrase (e.g. “I got promoted, I’m now a principal analyst.” vs. “The user’s job title is principal analyst.”—cosine falls under the filter’s

threshold). The reader then sees several structurally-identical, unordered conflicting statements with no recency cue and often answers from a stale one. Mem0-style—which *deletes* superseded memories outright rather than retiring them into a still-recallable raw pool—has nothing to leak from and stays flat through $U=32$; naive RAG and HippoRAG-v2-style, with no consolidation at all, collapse $100 \rightarrow 30$ from $U=8 \rightarrow 32$ exactly as §6.1 predicts. Diagnosed in this work; the fix and its measured effect follow immediately below (full curve, misses, and reproduction command in docs/BENCHMARK.md §9).

6.9 The fix: read-time belief-evidence consistency + currency-structured context

The diagnosis names two gaps: §4.3’s `_STALE_ECHO` filter is a *global* cosine bar tuned for near-verbatim echoes, and the reader prompt gives no recency cue over an unordered pool. Two additive, LLM-free, read-time fixes—the store and every ChurnBench ingestion cache are untouched, so no re-ingestion is needed. **(1) Read-time belief-evidence consistency:** narrower and more sensitive than the global filter, a raw slice is dropped only if it is close to a superseded fact whose *key already has a current fact in the top-k pool*—so the threshold can drop below the global filter’s without opening a cross-key false-positive surface. Current-fact ranking is byte-identical on or off; only raw-slice pool membership changes. Threshold chosen from a sweep on real $U=2$ data (ground truth via exact substring match against the deterministic attribute pools): 0.70 gives 100% recall on genuinely-stale echoes at a 7% false-positive rate on legitimately-current ones (0.60: 81% FPR; 0.80, the original global threshold: only 20.9% recall—why it missed this case). **(2) Currency-structured reader context:** the tenet arm’s context is split into “Current beliefs:” (distilled, dated) then “Supporting raw context:” (raw turns, dated) instead of one flat list—product-faithful, since the store already knows `is.current/valid.at` (the agent surface dates facts the same way); other arms have no such metadata, so their prompts are untouched by design.

U	tenet-baseline	tenet+1	tenet+2	tenet+1+2
2	60.0	90.0	82.0	98.0
8	42.0	84.0	82.0	92.0
32	36.0	80.0	70.0	82.0

Same cached stores across all four arms (only recall-time config differs), live **qwen3.7-plus** reader, $n=50$ /point, Wilson 95% CIs in docs/BENCHMARK.md §9.1. **Result: partial, pre-registered gate.** `tenet+1+2` clears $\geq 90\%$ at both $U=2$ (98.0) and $U=8$ (92.0) but falls short of Mem0-style’s flat 100 at $U=32$ (82.0)—the gate’s explicit “ship the fix, report half-life honestly” branch, not a full pass. **Churn half-life: `tenet+1+2` = 8**, up from < 2 , now tied with RAG and HippoRAG-v2-style, still behind Mem0-style’s 32 (immune by construction—it deletes superseded memories outright). All regression gates pass with the fix defaulted on (7 deterministic suites; §6.1’s $U=8$ stays 100%; FactConsolidation `sh_6k` is provably invariant—that arm never stores raw slices), so `recall()`’s `consistency_threshold` now defaults to 0.70 in shipped code.

A further write-side fix lifts the half-life to 32 (run-dependent). The read-time fix attacks stale raw evidence *after* it is stored; a complementary write-side fix removes it at the source. The embedding key-resolution of §8 (a conversational update supersedes a same-subject belief whose attribute-key embeds within 0.78 and whose text clears the 0.66 floor) makes paraphrased updates actually collide at ingestion, so far fewer stale versions ever enter the raw pool. With both

fixes on (the shipped default), a four-arm run at $n=30$ (Wilson CIs in `docs/BENCHMARK.md` §14) scores CHURNBENCH at 100/100/100 across $U \in \{2, 8, 32\}$ and dominates the retrieval baselines (RAG/HippoRAG-v2-style collapse to 30 at $U=32$); a separate run under the identical config (§9.1 in `docs/BENCHMARK.md`) scores 82 at $U=32$ instead of 100—**qwen3.7-plus** reader non-determinism on $n=30$ –50, not a configuration difference. **Honest ceiling claim: churn half-life 32, $U=32 \approx 82$ –100% across runs:** at best TENET *ties* delete-outright Mem0-style (flat 100) while keeping **LLM-free reads**, where Mem0-style pays an LLM add/update per write, but it does **not** beat Mem0-style on raw churn accuracy. The pre-registered falsification is closed on the half-life metric—by a mechanism the belief-state design predicts rather than a benchmark-specific patch—not by surpassing the idealized delete-outright arm.

6.10 Closing the write-path dependency: a local distiller

Every result above distills with Qwen Cloud (**qwen3.7-plus**); the write path—turning a message into keyed JSON facts—is TENET’s only remaining LLM dependency (reads are already LLM-free, §5). We probe whether a small model, LoRA-tuned and served fully offline, closes it: can a 1.5B model reproduce the reference distiller’s supersession behavior with zero cloud calls?

Method. LoRA SFT (bf16, rank 16, 2 epochs, ~ 490 synthetic message $\rightarrow$ facts pairs, assistant-only loss) of Qwen2.5-0.5B/1.5B-Instruct, labels from the cloud reference with keys *force-canonicalized* per known attribute (one key regardless of phrasing). An early candidate’s fabrication rate of 1.0 traced to a data-balance bug—only 6% of training examples had an empty target—not a capacity limit; rebalancing to $\sim 22\%$ empty-target examples (free to generate, no cloud labeling) dropped it to 0.08–0.17 with F1/key-consistency unchanged. Evaluated on a *decontaminated* held-out set: novel values and phrasings, 0/66 message overlap with training data (an earlier, contaminated screen inflated key-consistency by ~ 0.17 and hid the finding below entirely).

candidate	F1	fabrication	key-consist.	churn (superseded/6)
qwen2.5-0.5b-instruct (untuned)	0.295	0.333	0.30	0/6
qwen2.5-1.5b-instruct (untuned)	0.405	1.0	0.40	5/6
tenet-distiller-0.5b-v2 (LoRA)	0.867	0.167	0.75	3/6
tenet-distiller-1.5b-v2 (LoRA)	0.652	0.0	0.775	6/6
qwen3.7-plus (cloud reference)*	1.0	0.0	0.707	–

* measured on the contaminated screen, not the decontaminated set—reported for context, not as an apples-to-apples comparison.

Finding: key-consistency, not F1, is the load-bearing axis for supersession. The untuned baselines cannot supersede reliably (0/6, 5/6) despite non-trivial F1; the 0.5B LoRA variant has *higher* F1 than the 1.5B one (0.867 vs 0.652) but lower key-consistency (0.75 vs 0.775) and only 3/6 clean-churn supersessions—a 0.5B-specific out-of-distribution consistency gap the contaminated screen did not surface. The 1.5B LoRA model reproduces the reference’s supersession behavior fully offline (6/6, zero fabrication) and its canonical-key training *exceeds* the cloud reference’s own key-consistency (0.775 vs 0.707), because forcing one key per attribute at training time is a constraint ad hoc cloud prompting does not get for free. It passes an end-to-end supersession gate (move/manager-change) the untuned baselines fail, and ships as an opt-in LLM_PROVIDER=`ollama` path, making the full learn $\rightarrow$ supersede $\rightarrow$ doubt loop run with zero cloud calls. We report this as a probe, not a production claim: the eval is $n=26$ messages / 8 paraphrase groups, deterministic point estimates without confidence intervals—directionally strong enough to ship as opt-in, wide- N

validation is future work. Full tables and the LoRA training pipeline: `docs/BENCHMARK.md §10`, `scripts/distiller_lora/`.

7 Discussion

Where the belief state wins. Both MAB competencies reward what supersession provides: FactConsolidation directly (the store contains exactly one current value; no per-read freshness inference), EventQA indirectly (date-aware structured chunks preserve event order that flat chunking destroys). The churn benchmark isolates the mechanism: immunity is by construction, not tuning.

Where it loses. Multi-hop chaining over narrative text (RULER MH-QA) rewards explicit graph traversal; a keyed belief state has no edges to walk. Multi-session synthesis needs evidence from several sessions at once; expansion only deepens sessions already surfaced. Both are retrieval-topology limits, not consistency limits—graph-free was a deliberate trade (Mem0’s own ablation found graphs 3× slower for thin gains), and the gap quantifies what that trade costs.

Cost asymmetry. Among published systems above 50 on AR, TENET is the only one with LLM-free ingestion *and* LLM-free reads; HippoRAG-v2 spends LLM calls per ingested token, QwenLong-L1.5 spends 128K read tokens per query, ActMem builds LLM causal graphs at ingestion. At equal accuracy this asymmetry compounds over an agent’s lifetime: ingestion happens once per fact, reads happen forever, and TENET prices both at embedding cost.

Ingestion-time consistency makes read-time conflict resolution redundant. We tested this directly: adding CAR-style read-time `max(serial)` aggregation ([9]’s trick, the current FactConsolidation SOTA mechanism) as an optional reader over TENET’s pool changed nothing—FC multi-hop 15.0 → 15.0, LoCoMo 29.0 → 28.0 ($n=20/100$). Because supersession already keeps exactly one current value per key, there is rarely a duplicate-key group left for an aggregator to collapse; the work CAR does at every read, TENET does once at write. This is the belief-state thesis stated as a null result: the aggregation step that other systems need is redundant when the store is self-consistent by construction. (Retraction is the converse—an explicit “forget X ” that deletes without replacement is semantically distinct from value supersession; a tombstone op is implemented but stays opt-in, as removing a fact entirely hurt a multiple-choice benchmark that needs the retracted context to recognize the retraction.)

8 Limitations

Verbatim-recall benchmarks favor raw storage. On LoCoMo-10 ($n=500$ stratified, qwen-judged, same reader/judge both arms) naive RAG beats TENET 38.8 vs 33.8 (McNemar $p=0.031$), concentrated in temporal and single-hop questions whose gold answers reward the *exact wording* of a raw turn: distillation paraphrases that wording away. LoCoMo stresses verbatim multi-session recall, not knowledge churn; we report the loss plainly (BENCHMARK.md §12). A related audit finding: the published LoCoMo audit’s 156 error entries correct *evidence citations*, not gold answers—an “audit-corrected” key changes QA accuracy by exactly 0.0 pp by construction. **Natural-language updates under-fire keyed supersession.** On PersonaMem-v2 (NapMem’s benchmark; 4-way MC, $n=485$, deterministic scoring) a blind no-memory control scores 34.4% vs both memory arms at ~50.5%—so memory is load-bearing and the comparison is valid—but TENET and RAG *tie* (50.5 vs 50.9, McNemar $p=0.92$), and on the retraction subset we predicted TENET would win, RAG leads if anything (74.2 vs 67.7, ns). Cause, measured: only 3.8% of turns trigger keyed supersession because conversational “I’ve switched to X ” rarely produces a distilled key that collides with the prior belief—the paraphrase gap again. *We then fixed this:* an LLM-free embedding key-resolution

(a new fact supersedes a same-subject current fact whose attribute-key embedding is cosine ≥ 0.78 and value-compatible, with the fact-text similarity floor raised to 0.66 to reject shared-salient-word coincidences) plus a distiller constrained away from vague umbrella keys raises true-fire on an expanded labeled set (including adversarial shared-word negatives) from 40% to 80% at 0% false-fire, lifts real PersonaMem firing $3.8\% \rightarrow \sim 7\%$ and overall accuracy $50.5 \rightarrow 53.4$, and passes every regression gate (default on, hardened). It does *not* fully address explicit retractions (“forget X ”), which are deletions, not value replacements—a tombstone mechanism is the separate next step. This bounds our churn claim: it holds when updates carry a stable attribute (§6.1, §6.4), and now degrades gracefully rather than sharply on free-form preference drift. (We repurposed PersonaMem’s full-context task as retrieval, so absolute scores are not vendor-comparable; BENCHMARK.md §13.) **Multi-session synthesis** still trails RAG (42.9 vs 57.1) under gpt-4o-tier readers—though this inverts under 2026 frontier readers (§: reader-generality); session-diverse retrieval is the natural next step. **Multi-hop chaining** trails graph memory (45 vs 66). **The frontier is a knob, not a free lunch**: parity accuracy costs RAG-equal tokens; the $1.6\times$ per-token win trades ~ 5 pp of one-shot accuracy. **Evaluation scale**: LongMemEval numbers are $n=40$, one seed, off-Qwen readers (reader noise $\approx \pm 5\text{--}7$ pp; hence “parity”, not a win); MAB numbers are full-scale (800 and $\sim 2,000$ questions) with CIs. **Distillation quality**: in free-form conversation, keys come from a small LLM distiller; a missed key match degrades supersession to ordinary storage (fail-soft, but the churn guarantee is only as good as the keying). **Scope**: single-user assistant memory; multi-writer stores and adversarial updates are untested. **Stale raw-turn leakage under natural conversational churn** (§6.8, measured, not hypothetical): with several attributes each updated many times via paraphrased statements, already-superseded raw turns can survive the write-time stale-echo filter and outvote the correct current fact in the reader’s context. **Fixed, with a caveat** (§6.9): read-time belief-evidence consistency plus a currency-structured reader context raise churn half-life from < 2 to 8 on their own (both regression-gate-clean, defaulted on); adding write-side embedding key-resolution lifts it further to 32 ($U=32 \approx 82\text{--}100\%$ across runs, reader non-determinism). At best this *ties* Mem0-style’s write-time-consolidation immunity at $U=32$ (flat 100)—it does not surpass it on raw accuracy, only match it while keeping LLM-free reads—so the failure mode is closed on the half-life metric, not strictly dominated.

9 Conclusion

Treating agent memory as a self-consistent belief state rather than a document index makes it robust to the way real knowledge behaves: it changes. TENET stays correct under knowledge churn where retrieval memory structurally collapses, exceeds the published single-hop conflict-resolution SOTA at its reader tier on a weaker backbone, and is the only published-competitive system on MemoryAgentBench whose ingestion and reads are both LLM-free. The belief-state view also yields time-travel and principled forgetting for free. The LLM-free read path is also cheap at scale: with a resident embedding matrix (built once, appended incrementally, matmul at query time) recall stays ~ 11 ms at 10^5 stored facts and extrapolates to ~ 24 ms at 10^6 (versus > 1 s and ~ 10 s for a reload-per-query implementation), so RAM—not latency—is the eventual bound (docs/SCALE.md). We hope knowledge churn becomes a standard axis for evaluating agent memory.

Reproducibility. All code, harnesses, official-prompt copies, cached ingestion artifacts, and per-question dumps: github.com/Nas01010101/tenet (docs/BENCHMARK.md lists one command per reported number).

References

- [1] P. Chhikara et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413, 2025.
- [2] D. Wu et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813, 2024.
- [3] D. Wu et al. LongMemEval-V2: Evaluating Long-Term Agent Memory Toward Experienced Colleagues. arXiv:2605.12493, 2026.
- [4] P. Rasmussen et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956, 2025.
- [5] C. Packer et al. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.
- [6] W. Xu et al. A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110, 2025.
- [7] Y. Hu et al. MemoryAgentBench: Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. arXiv:2507.05257, ICLR 2026.
- [8] B. Gutiérrez et al. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models (HippoRAG 2). arXiv:2502.14802, 2025.
- [9] Don't Ask the LLM to Track Freshness: A Deterministic Recipe for Memory Conflict Resolution. arXiv:2606.01435, 2026.
- [10] Temporal Validity in Retrieval Memory: Eliminating Stale-Fact Errors for AI Agents over Evolving Knowledge. arXiv:2606.26511, 2026.
- [11] Less Context, More Accuracy: A Bi-Temporal Memory Engine for LLM Agents. arXiv:2606.09900, 2026.
- [12] TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory. arXiv:2606.06240, 2026.
- [13] Shen et al. QwenLong-L1.5: Post-Training Recipe for Long-Context Reasoning and Memory Management. arXiv:2512.12967, 2026.
- [14] Yu et al. Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for LLM Agents. arXiv:2601.01885, ACL 2026.
- [15] Zhang et al. ActMem: Bridging the Gap Between Memory Retrieval and Reasoning in LLM Agents. arXiv:2603.00026, 2026.
- [16] MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent. arXiv:2507.02259, 2025.
- [17] NapMem: Memory as a Structured Action Space for Long-Horizon Agents. arXiv:2607.05794, 2026. Qwen Large Model Application Team, Alibaba.
- [18] MemFlow: Training-Free Intent-Routed Memory Orchestration with LLM Validation. arXiv:2605.03312, 2026.
- [19] MemReader: Active Extraction for Agent Memory Construction. arXiv:2604.07877, 2026.
- [20] Mem- α : Reinforcement-Learned Memory Construction for LLM Agents. OpenReview, ICLR 2026.
- [21] MemCog: Cognitively-Inspired Active Navigation of Agent Memory. arXiv:2605.28046, 2026. (Affiliation unverified.)
- [22] K. Friston. The free-energy principle: a unified brain theory? *Nat. Rev. Neurosci.*, 2010.

A Protocol details

SubEM normalization and the reader prompt for FactConsolidation are copied verbatim from the MemoryAgentBench repository. **LongMemEval(S*)** judging uses the five official anscheck templates (including the abstention template) verbatim; judge failures exclude the instance from scoring. **EventQA** uses a copy-one-candidate reader with the official choice sets. **Caching**: ingestion caches are keyed by the MD5 of the exact chunk list, and a store/chunk length assertion guards against stale-cache pairing. **Controls**: naive-RAG shares the embedder, chunker, reader, prompt, and k with TENET in every table.

B Reproduction commands

FactConsolidation: `LLM_PROVIDER=ollama OLLAMA_MODEL=qwen2.5:7b EMBED_PROVIDER=local python scripts/bench_factcon.py --qpc 100 --tenet-read decompose --keys heuristic`. Accurate Retrieval: `LLM_PROVIDER=openrouter OPENROUTER_MODEL=openai/gpt-4o-mini JUDGE_PROVIDER=openrouter JUDGE_MODEL=openai/gpt-4o EMBED_PROVIDER=local python scripts/bench_mab_ar.py --qpc 100 --judge`. Churn: `python scripts/bench_horizon.py --principals 12 --k 6 --updates 2,4,6,8,10,12`. Frontier: `python scripts/lme_recall.py --limit 40 --k 10 --qa --seed 2 [--expand 20]`. Ablations: `python scripts/bench_knowledge_update.py --principals 4`. Baseline arms: `python scripts/bench_baselines.py --arms car,mem0,hipporag,memagent`.